

AIRBNB SYSTEM DESIGN

- DECIDING REQUIREMENTS
 - 1.Functional Requirements
 - 2.Non Functional Requirements
- CAPACITY ESTIMATION
 - 3.DAU-MAU
 - 4.Throughput
 - 5.Storage
 - 6.Memory
 - 7.Network and Bandwidth Estimation
- API DESIGN
 - 8.API Design:Add Property
 - 9.API Design:View Bookings
 - 10.API Design:Search Properties
 - 11.API Design:View Property
 - 12.API Design:Book Property
- HIGH LEVEL DESIGN
 - 13.HIGH LEVEL Design:Add Property
 - 14.HIGH LEVEL Design:View Bookings
 - 15.HIGH LEVEL Design:Search Properties
 - 16.HIGH LEVEL Design:View Property
 - 17.HIGH LEVEL Design:View Bookings

- DEEP DIVE INSIGHTS
 - 18.Deep Dive Insights:Database Selection
 - 19.Deep Dive Insights:Data Modeling
 - 20.Deep Dive Insights:Handling Concurrent Booking Updates
-

DECIDING REQUIREMENTS

Functional Requirements

There will be two types of users:

- **Property Owners:** Users who own the property
- **Guests:** Users who rent the property

Functional Requirements for Property Owners

Requirement	Description
Add/Update Property	Property owners can add or update their properties. This includes setting the property's availability, description, images, location, and price.
View Bookings	Property owners can view all their bookings.

Functional Requirements for Guests

Requirement	Description
Search Properties	Guests can search for properties based on various criteria such as location and price.
View Properties	After searching, guests can see a list of properties. They can click on any property to view more detailed information.
Book Properties	Guests can book a property of their choice.

Requirement	Description
View Bookings	Guests can view their own booking history.

Non-Functional Requirements

Non-Functional Requirements for Property Owners

Requirement	Description
Availability	The system should be highly available with 99.999% uptime.
Scalability	The system should scale up smoothly even when more properties are added and more people use it.

Non-Functional Requirements for Guests

Requirement	Description
Availability	The system should be highly available with 99.999% uptime.
Strong Consistency	When a guest books a property, no one else should be able to book that same property for the same time. In other words, bookings should have strong consistency.
Moderate Latency	It's okay if booking takes a few seconds, but it should still happen fairly quickly.

CAPACITY ESTIMATION

DAU MAU

How many users will be using your software in a single day?

Metric	Description	Value
DAU (Daily Active Users)	The number of users actively using the software in a single day.	10 million
MAU (Monthly Active Users)	The number of users actively using the software over a month.	100 million

Throughput

Writes

If we look at the functional requirements, there are only two main ways to write data into the system:

- 1. **Property Owners create or update property listings.**
This creates/updates property data in the system.
- 2. **Guests book a property.**
This creates booking data in the system.

Most of the users on the platform are guests, and property owners are relatively fewer. We can make a reasonable assumption that **1 out of every 50 users is a property owner.**

We know that the platform has **10 million Daily Active Users (DAU):**

- **Property owners** = $(1/50) \times 10 \text{ million} = 200,000 \text{ owners}$
- **Remaining 9.8 million are guests**

Assumptions

- Every property owner creates or updates a property once a week.
- Every guest books a property once a month.

Write Throughput Calculation

Metric	Calculation	Result
Create/Update Requests per Day	200,000 owners × (1/7 requests per day)	28,571 create/update requests per day
Booking Requests per Day	9.8 million guests × (1/30 booking requests per day)	326,667 booking requests per day
Total Write Requests per Day	28,571 (create/update) + 326,667 (booking)	0.35 million write requests per day

Storage

Property Data

Let’s assume each property listing (including images, description, and metadata) requires:

- **Property Details:** 10 KB
- **Images:** 450 KB
- **Metadata:** 40 KB

Total Storage per Property

10 KB + 450 KB + 40 KB = 500 KB

Daily Storage Requirement

From our throughput estimation, we know that **28,571** new property create/update requests are made per day.

Total Storage per Day = 500 KB × 28,571 requests/day = 14.29 GB/day

10-Year Storage Requirement

14.29 GB/day × 365 days/year × 10 years = 52.15 TB for property data.

Booking Data

Bookings are much smaller compared to property data but still essential. Each booking includes information like dates, guest info, and property details.

Storage per Booking = 1 KB per booking

Daily Storage Requirement

From our throughput estimation, we know there are **326,667** booking requests per day.

Total Storage per Day = 1 KB × 326,667 requests/day = 326.67 MB/day

10-Year Storage Requirement

326.67 MB/day × 365 days/year × 10 years = 1.19 TB for booking data.

Summary

- **Total Daily Storage Requirement:** 14.29 GB + 326.67 MB ≈ **14.61 GB/day**
 - **Total 10-Year Storage Requirement:**
 - Property Data: **52.15 TB**
 - Booking Data: **1.19 TB**
-

Memory

Definition

By **memory**, we mean **cache memory size**.

Importance

Accessing data from a database takes a long time. To access data faster, we use **cache**.

Example Calculation

Let's assume we need to cache **5% of the daily data**:

- **Daily data size:** 14.61 GB/day
- **Cache requirement:** 5% of 14.61 GB/day
- **Result:** **0.7305 GB/day**

The memory size should also **scale as the system grows** to meet the increasing data demands.

Network and Bandwidth

Data Flow Into the System (Ingress)

This represents the amount of incoming data into the system, which eventually gets stored.

So, the total incoming data will be equal to the amount of data we store in a day.

Daily Storage Requirement

- **Property Data:** 14.29 GB/day
- **Booking Data:** 326.67 MB/day
- **Total:** 14.61 GB/day

Thus, **total incoming data per day = 14.61 GB/day.**

Calculation for Ingress

Total incoming data per second (Ingress) =
(14.61GB/day ÷ (24 × 60 × 60) ≈ 169.1 , KB/s)

Data Flow Out of the System (Egress)

The data is mainly read by two operations: searching for properties and viewing property details.

Other operations, like viewing bookings, have a much smaller data size (around 1 KB per request). They also have lower throughput. Therefore, we can safely ignore them for outgoing data calculation.

Search Requests

- Assume each search request returns **10 properties.**

- Average size of one property listing: **500 KB**

Data per search request = 10 properties × 500 KB = 5 MB

From the quiz, there are **2.94 million search requests per day**.

Total data read for search requests = 2.94 million × 5 MB = 14.7 TB/day

View Property Requests

- Average size of each property listing: **500 KB**
- Number of property views per day: **5.88 million**

Total data read for viewing properties = 5.88 million × 500 KB = **2.94 TB/day**

Total Egress Calculation

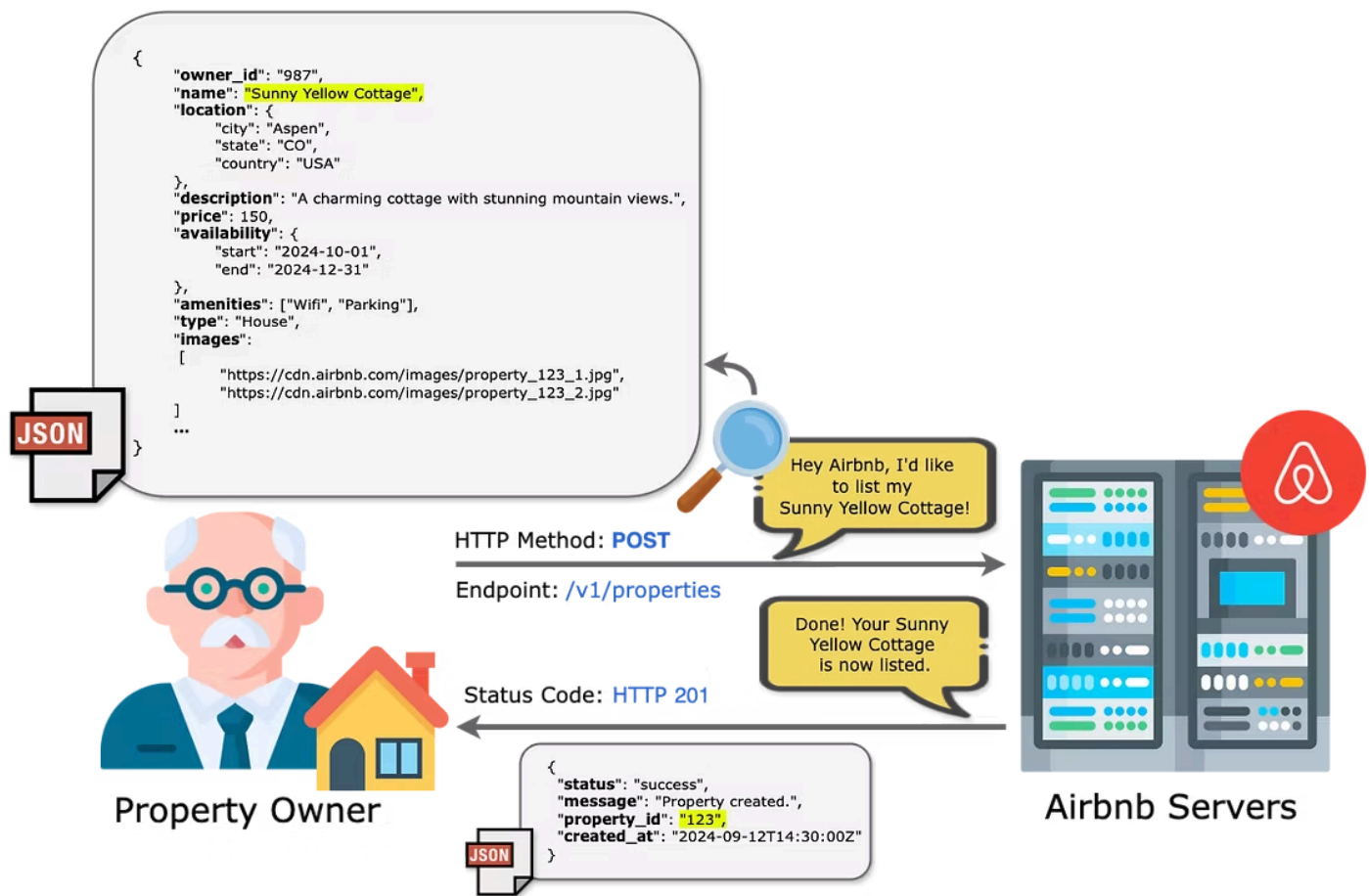
- Total outgoing data per day = **14.7 TB/day + 2.94 TB/day = 17.64 TB/day**
 - Total outgoing data per second (Egress) = **17.64 TB ÷ (24 × 60 × 60) ≈ 204.2 MB/s**
-

API DESIGN

API DESIGN :Add Property

When the property owner asks the server to create a new property listing on the website, he uses an API call.

Precisely, a REST API is used for the communication. Here are the technical details:



HTTP Method

This tells the server what action to perform. Since we want to create something new on the server (a property listing), we will use the **POST** action.

Endpoint

This tells the server where to perform that action. Since we are creating a property listing, we will send our request to the **/v1/properties** endpoint of the server.

Note

v1 means version 1. It is a good practice to version your APIs.

HTTP Body

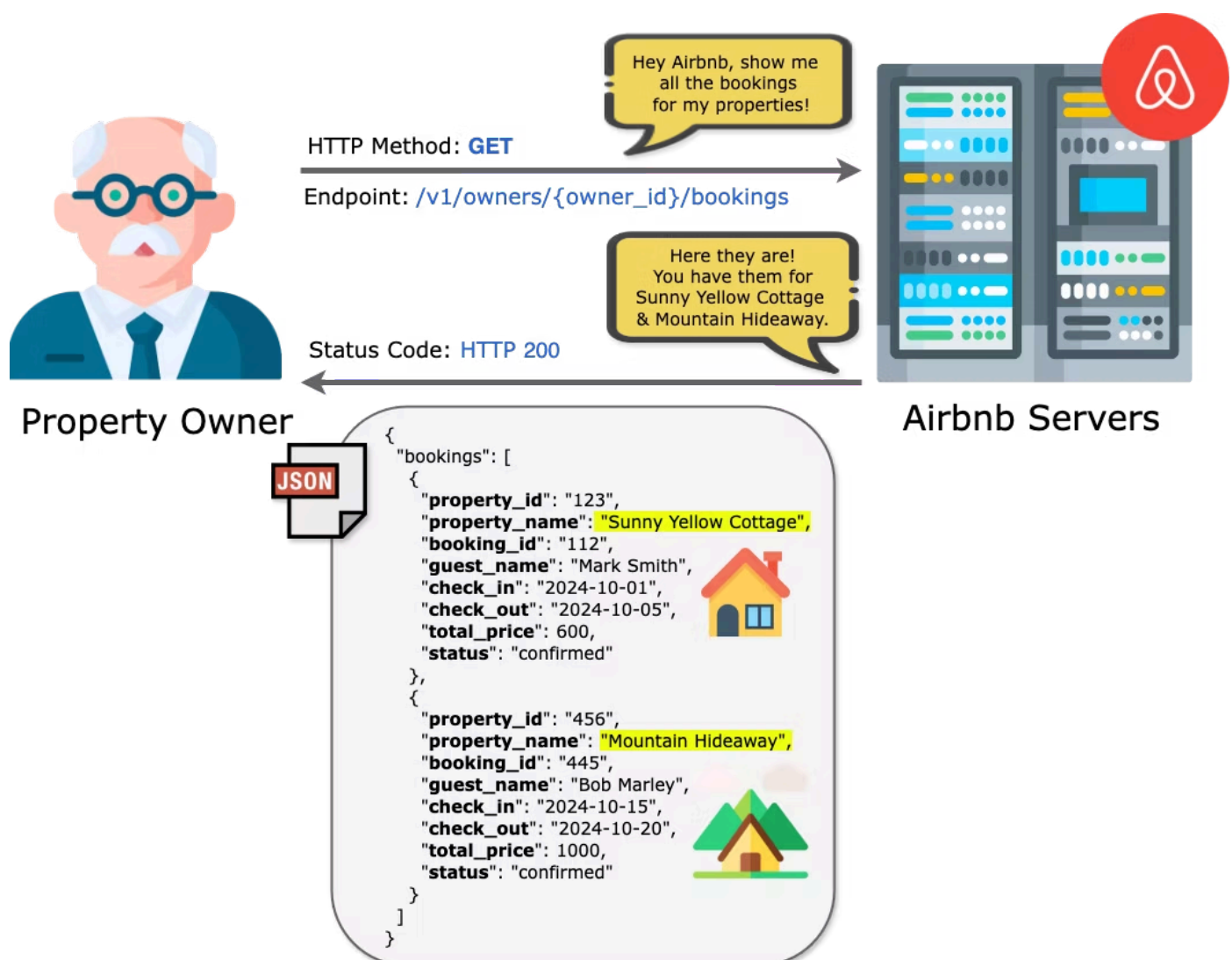
We've told the server we want to create a property listing, but we haven't yet provided the details of the property itself. This information is sent in the request body.

- **owner_id**: The ID of the property owner
- **name**: The name of the property
- **location**: The location of the property (city, state, country)
- **description**: A short description of the property
- **price**: The nightly price to rent the property
- **availability**: The dates when the property is available for rent
- **amenities**: What features does the property offer (e.g., Wifi, Parking)
- **images**: The associated images of that property
- ...

API DESIGN :View Bookings

When a property owner wants to see all their bookings, they send an API request to Airbnb's servers.

Precisely, a REST API is used for the communication. Here are the technical details:



HTTP Method

This tells the server what action to perform. Since the owner is retrieving booking data (not creating it), we will use the **GET** method.

Endpoint

This tells the server where to perform the action. Since we are viewing bookings for a specific owner, we will send the request to the **/v1/owners/{owner_id}/bookings** endpoint. **{owner_id}** is a placeholder for the actual owner's ID. The server uses this ID to get all bookings for properties owned by that owner.

Note

GET requests do not include a body because they are used to fetch information, not to send data.

Response

After sending the request, the server processes it and returns a list of bookings for all properties owned by the owner.

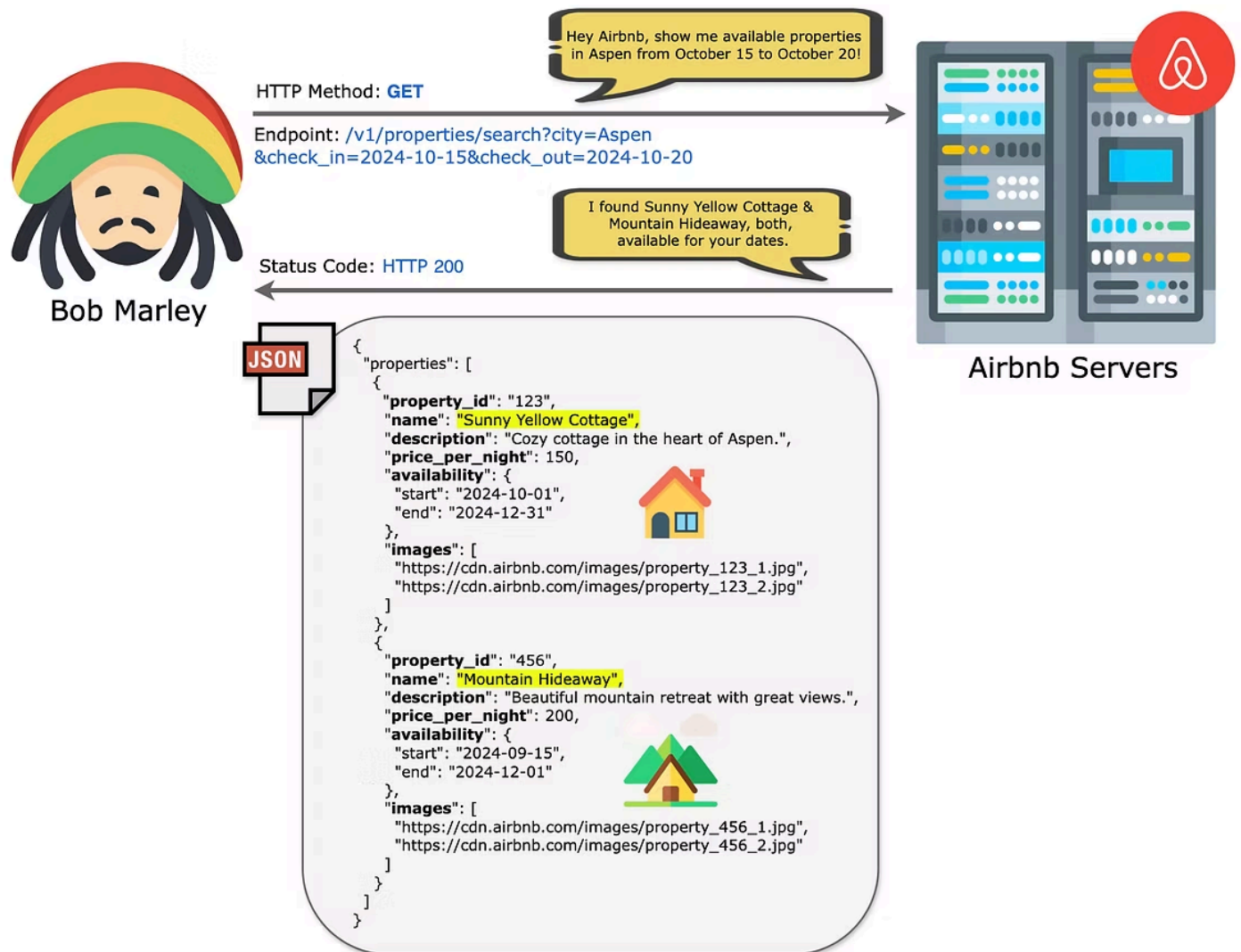
The response includes the following details:

- **property_id:** The unique ID of the property.
- **property_name:** The name of the property.
- **booking_id:** The unique ID of each booking.
- **guest_name:** Name of the guest who made the booking.
- **check_in** and **check_out:** Dates of the guest's stay.
- **total_price:** Total cost of the booking.
- **status:** The status of the booking (e.g., confirmed).

API DESIGN :Search Properties

When a guest wants to search for available properties, they send an API request to Airbnb's servers.

We will use a REST API for this communication. Here are the technical details:



HTTP Method

This tells the server what action to perform. Since we want to 'get' all the properties based on our search, we will use the **GET** method.

Endpoint

This tells the server where to perform that action. Here, the guest wants to search for properties so we will send our request to the `/v1/properties/search` endpoint. We will also use appropriate query parameters to provide:

- **city:** location where the guest wants to search.
- **check_in & check_out:** The dates for which the guest needs the property.

Example

`/v1/properties/search?city=Aspen&check_in=2024-10-15&check_out=2024-10-20`

Note

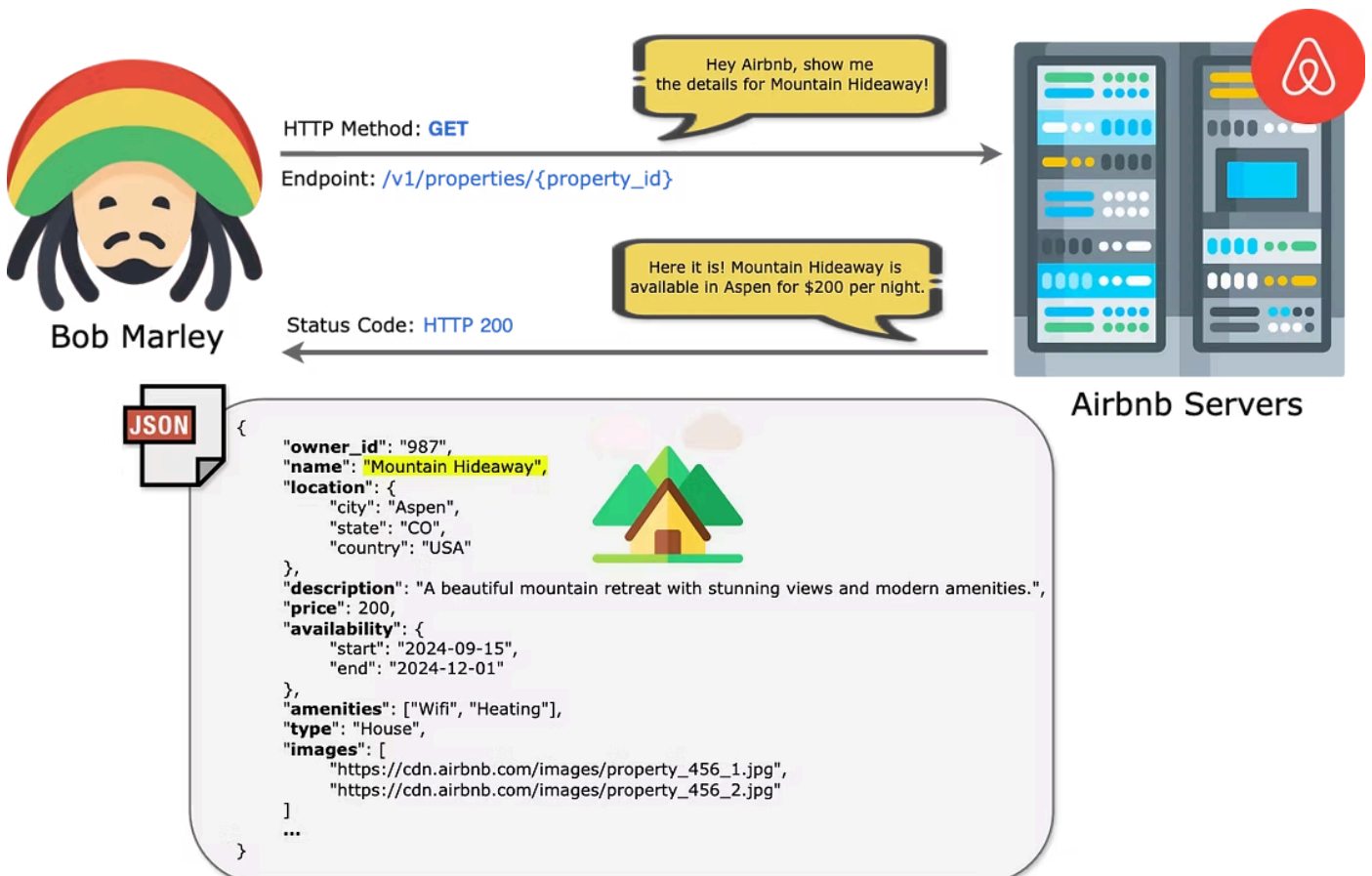
GET requests do not include a body because they are used to fetch information, not to send data. The information about the search (like city and dates) is passed as query parameters in the URL.

After sending the request, the server processes it and returns a list of available properties based on the search criteria.

API DESIGN: View Property

When a guest wants to view the details of a specific property, they send an API request to Airbnb's servers.

We will use a REST API for this communication. Here are the technical details:



HTTP Method

This tells the server what action to perform. Since the guest is 'getting' the property details, we use the **GET** method.

Endpoint

This tells the server where to perform that action. The guest wants to view details of a specific property, so the request is sent to the **/v1/properties/{property_id}** endpoint, where **{property_id}** is the unique ID of the property.

Note

GET requests do not include a body because they are used to fetch information, not to send data.

After sending the request, the server processes it and returns the full details of the property.

API DESIGN :Book Property

When a guest books a property, the process happens in three steps. In every step, we use REST APIs.

Let's look at these stages:

1. Property Selection:

The guest visits the property page, picks the dates, and clicks the 'Book' button.

2. Payment Process:

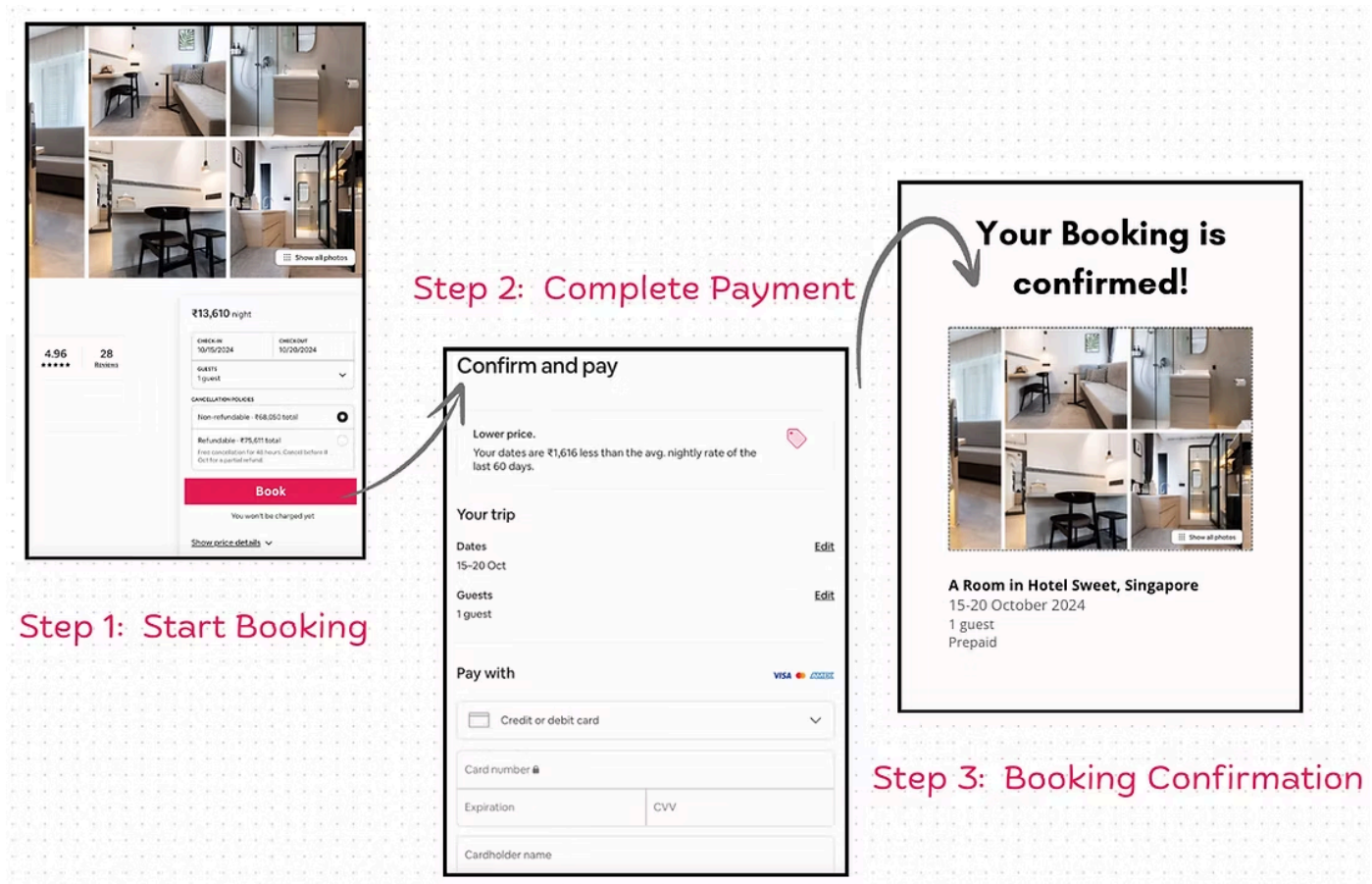
A payment page shows up, and the guest enters their payment details.

They can pay with a credit card or through services like PayPal.

If they choose PayPal, they get redirected to PayPal's site for a moment to complete the payment.

3. Booking Confirmation:

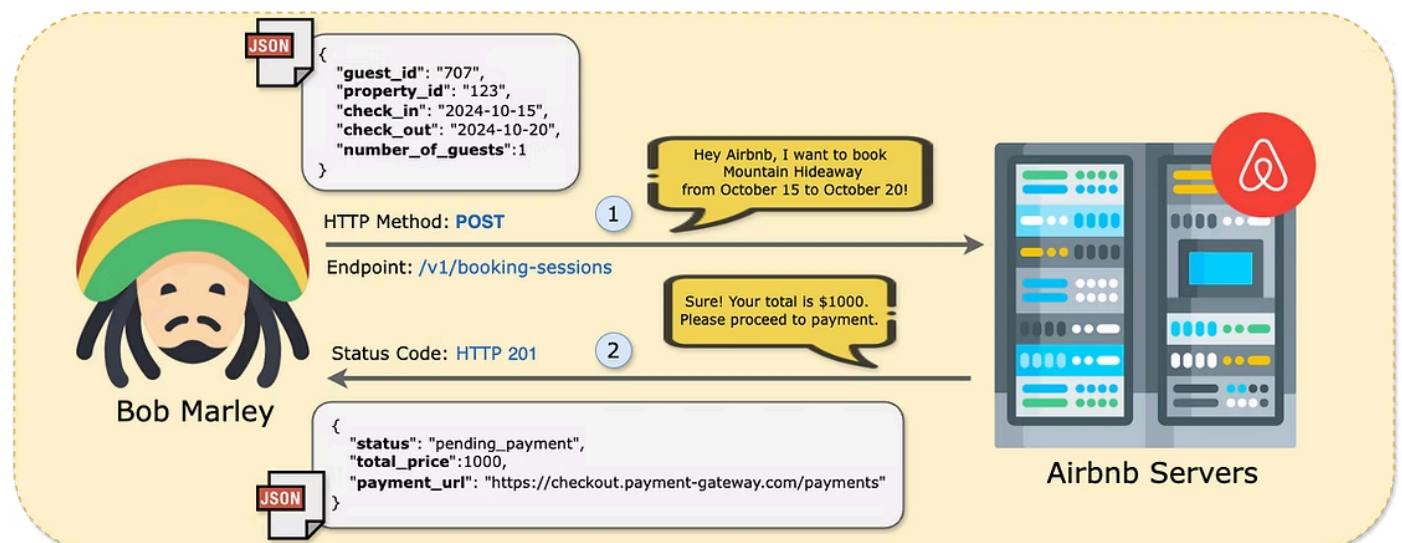
Airbnb checks if the payment was successful.
If yes, it confirms the booking.



Now, let's dive into how this works with detailed APIs step by step:

Stage 1: Guest Visits Property Page, Chooses Dates, Clicks the Book Button (Step 1 + Step 2)

The guest starts the booking process by creating a booking session. The actual booking will happen later, in Stage 3, after the payment is successfully completed in Stage 2.



HTTP Method

This tells the server what action to perform. Since we want to create a new booking session, we use the **POST** action.

Endpoint

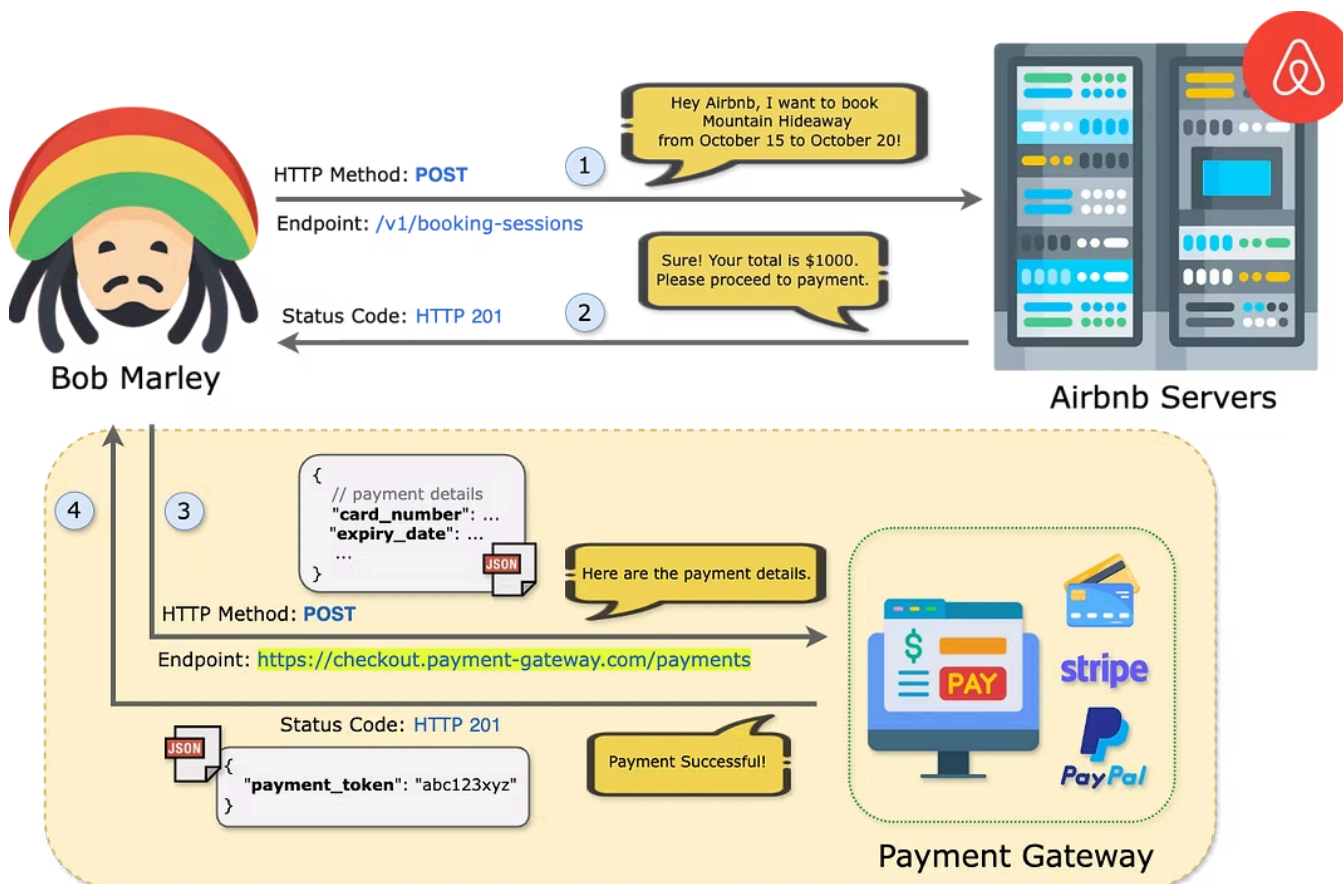
This tells the server where to perform that action. The request is sent to the **/v1/booking-sessions** endpoint.

HTTP Body

Here, we send all the necessary details about the booking—such as which property, the guest making the booking, how many guests, and the stay dates.

Airbnb responds back and redirects the client to a payment URL, asking the guest to complete the payment. That URL is the endpoint of the payment gateway, which is used to securely process payments from the client.

Stage 2: Guest Completes the Payment (Step 3 + Step 4)



HTTP Method

The client uses the **POST** action to send the payment information to the Payment Gateway.

Endpoint

The client uses the payment gateway URL received in the last stage:

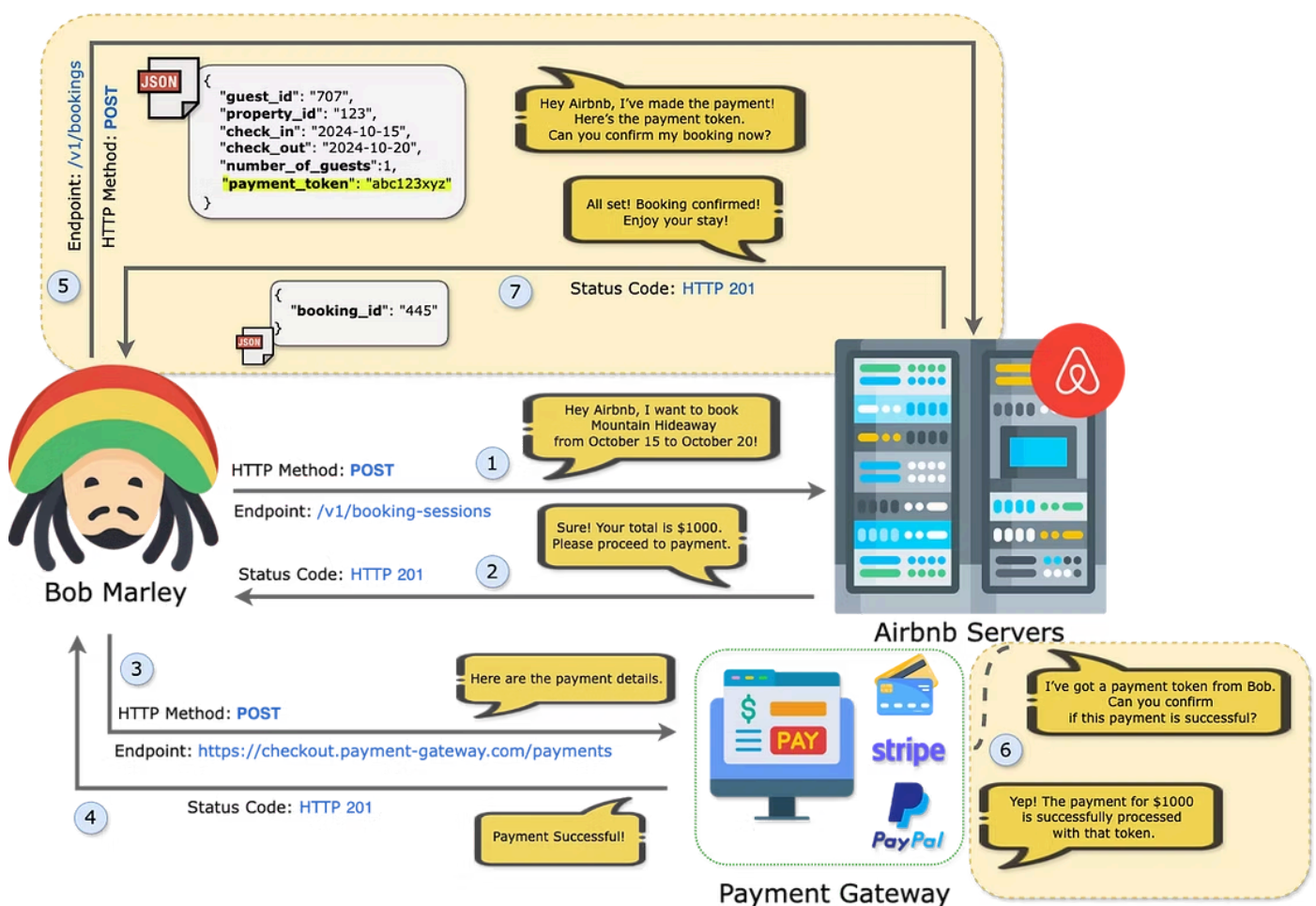
<https://checkout.payment-gateway.com/payments>

HTTP Body

The payment details are provided in the request body.

The Payment Gateway securely processes the payment and sends back a payment token, which acts as proof of the transaction.

Stage 3: Airbnb Confirms the Booking (Step 5 + Step 6 + Step 7)



- **Step 5:** The actual booking happens in this stage because the guest has already made the payment. The guest sends the proof of the payment (payment token received in the last step) to Airbnb.

- **Step 6:** Airbnb takes this token and double-checks with the Payment Gateway to see if the payment linked to this token has been completed successfully.
- **Step 7:** Once the Payment Gateway gives confirmation, Airbnb finalizes the booking and sends the booking ID to the guest for reference.

HTTP Method

The guest sends the booking details, along with the payment token, to Airbnb using the **POST** method.

Endpoint

Since we are creating the actual booking this time, we send the request to the **/v1/bookings** endpoint.

HTTP Body

Here, we send all the necessary details about the booking—such as which property, the guest making the booking, how many guests, and the stay dates. Along with this, we also send the payment token as proof of payment.

HIGH LEVEL DESIGN

HIGH LEVEL DESIGN :Add Property

Let's walk through how the high-level design works when a property owner adds (lists) a property on Airbnb.

Property Listing Process

1. Publish Property:

When the property owner clicks the **Publish** button to list their property, a **POST** request is sent to the **API Gateway**.

2. Request Body:

The request body contains all the property details, which have already been defined

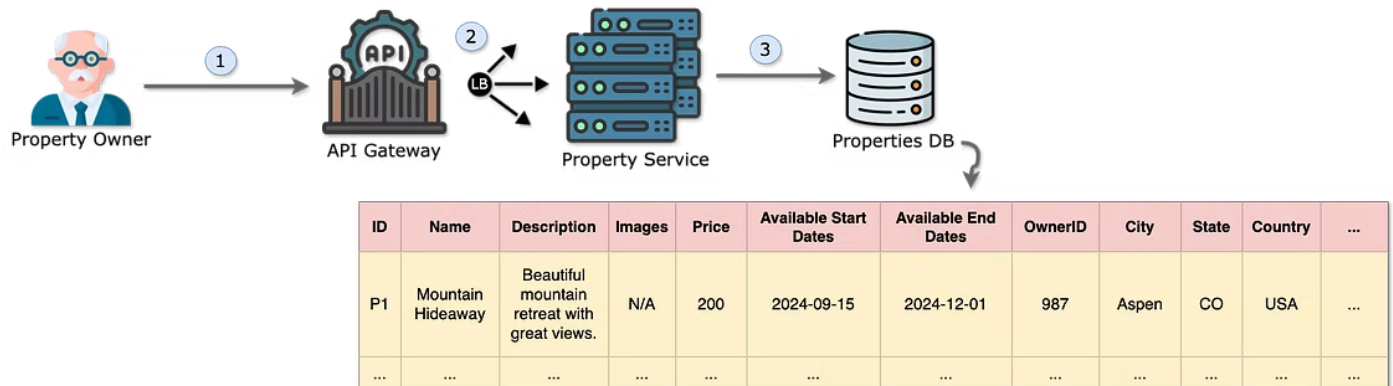
in the API Design section.

3. Request Routing:

The **API Gateway** routes this request to the **Property Service** via a **Load Balancer**.

4. Database Entry:

The **Property Service** adds the new property entry into the **Properties DB**.



This is pretty straightforward, right?

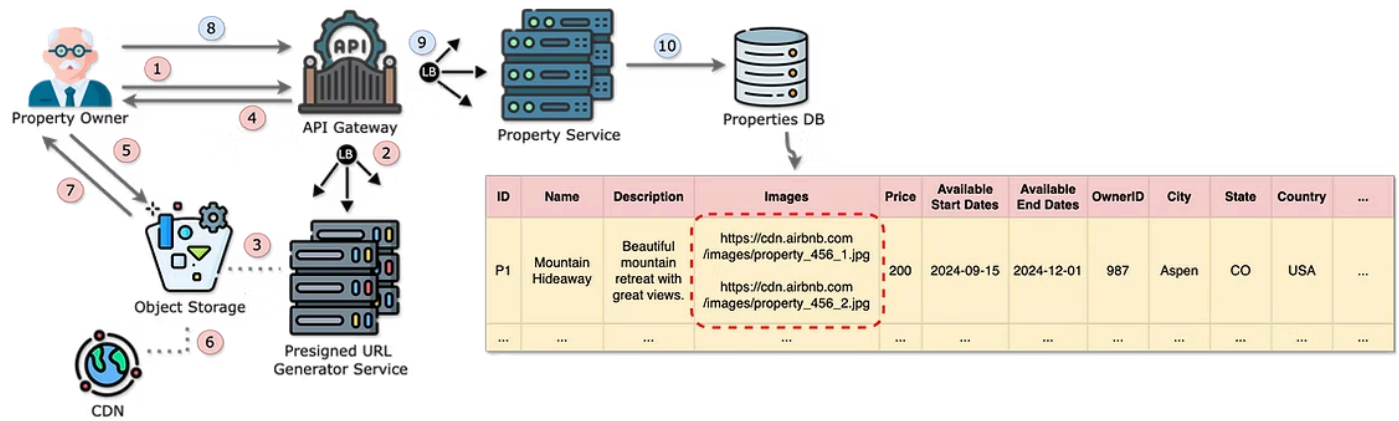
However, so far, we haven't included property images in this flow. If you look at the data model, you'll notice that images are missing.

Property Image Upload Flow

In reality, it's natural for properties to include images. Let's see how the flow looks in that case. The flow is mostly the same, but a few extra steps are added at the beginning.

Flow Overview

Here's what happens: When the property owner selects images from their device to upload to Airbnb App, behind the scenes, these images are actually uploaded to Airbnb's server. Let's understand this with a diagram:



1. Image Selection and Pre-Signed URL Request

- The property owner selects images and uploads them.
- A request is sent to the **API Gateway** asking for **Pre-Signed URLs** to upload the images.
- **Pre-Signed URL**: A special link that allows the property owner to upload images directly to **Object Storage**.

2. Request Routing

- The **API Gateway** routes this request to the **Pre-Signed URL Generator Service** via the **Load Balancer**.

3. Pre-Signed URL Generation

- The **Pre-Signed URL Generator Service** communicates with **Object Storage** and generates Pre-Signed URLs for each image.

4. URL Delivery

- These URLs are returned to the property owner.

5. Image Upload

- The property owner uses these URLs to upload the images directly to **Object Storage**.

6. Content Delivery

- **Object Storage** pushes these images to a **CDN (Content Delivery Network)** to enable fast access from anywhere in the world.

7. Image URLs Returned

- After uploading the images, the URLs of the uploaded images are returned to the property owner.

8. Publishing the Property

- The property owner clicks the **Publish** button to list the property.
- A **POST request** is sent to the **API Gateway**, which includes:
 - Property details
 - Uploaded image URLs

9. Property Entry Creation

- The **API Gateway** routes the request to the **Property Service** via the **Load Balancer**.
- The **Property Service** adds the new property entry, including the image URLs, to the **Properties DB**.

10. Database Update

- The image URLs are now stored in the **images** column of the property table.

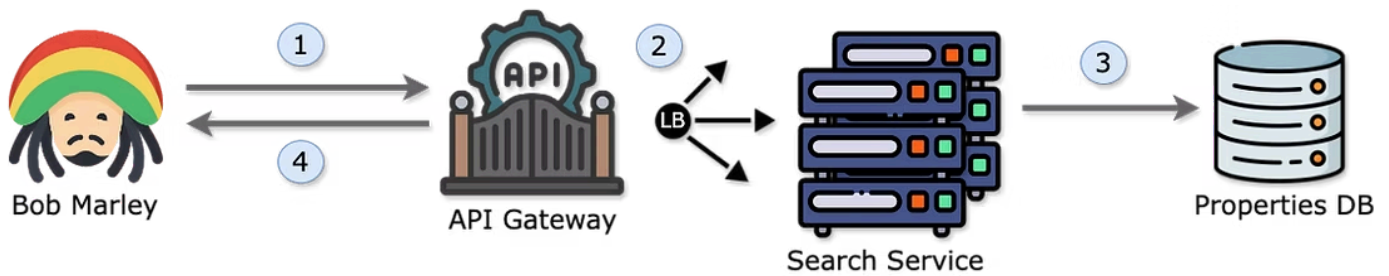
HIGH LEVEL DESIGN :Search Properties

Let's now take a look at the high-level design for when a guest searches for properties on Airbnb:

Search Process

1. When a guest enters details like check-in, check-out dates, location, price, and possibly some keywords (for example, "mountain view"), and hits the search button, a request is sent to the **API Gateway**.
2. The **API Gateway** forwards this request to the **Search Service** via a **Load Balancer**.

3. The **Search Service** queries the **Properties DB** for available properties based on the guest's preferences.
4. Search results are returned back to the guest.



Issues with this Approach

This approach works, but it's not very efficient. Why?

- Searching the **Properties DB** using multiple criteria like dates, location, or keywords can be slow.
- The **Properties DB** is structured in a simple table format, which isn't great for handling complex searches.
- Handling typos is problematic. For instance, a guest typing "montain view" instead of "mountain view" could miss out on valid results. Ideally, our search should automatically correct these errors and find similar matches (called **Fuzzy Search**). However, the **Properties DB** can't do this.

So, What's the Solution?

We use **Elastic Search** for this! Elastic Search is designed to handle complex queries. It supports features like:

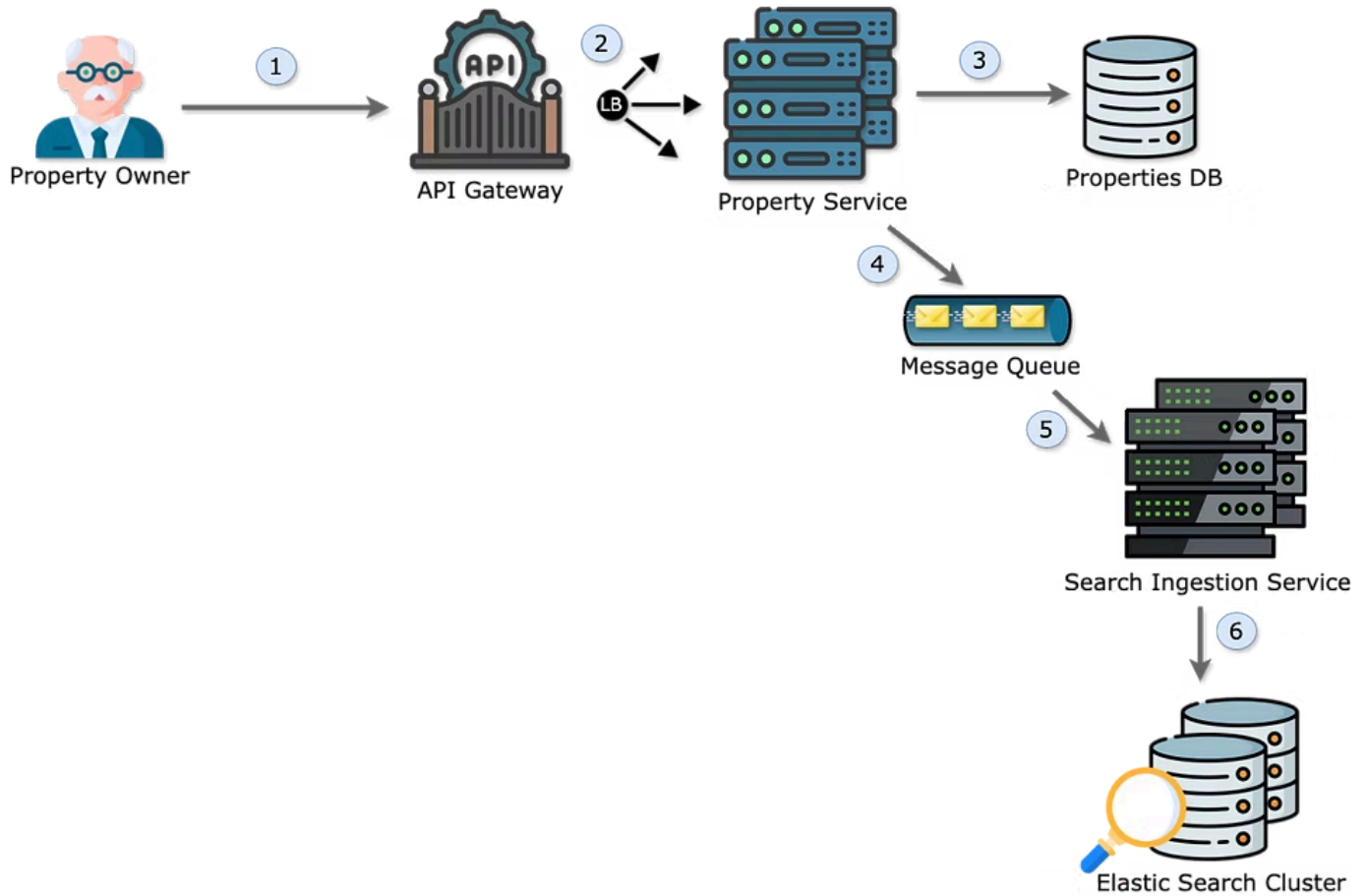
- **Range Queries:** e.g., for check-in and check-out dates
- **Filters:** for location, price
- **Fuzzy Search:** to handle misspelled keywords

Data Flow to Elastic Search

Whenever a new property is added to the **Properties DB**, we'll also send that data to **Elastic Search**. This allows us to quickly find results when a guest searches for properties.

Steps Breakdown:

We already understand steps 1, 2, and 3 from the previous lesson. Now, we'll add steps 4, 5, and 6 to show how property data is sent to Elastic Search.



1. Property Publish Request:

When the property owner hits the publish button to list their property, a **POST request** is sent to the **API Gateway**.

2. Forwarding the Request:

The **API Gateway** forwards this request to the **Property Service** through a **Load Balancer**.

3. Adding Property to Database:

The **Property Service** adds this new property to the **Properties DB**. But it doesn't stop here!

4. Creating an Event:

The **Property Service** creates an event with all the property details and sends it to a **Message Queue**.

5. Event Processing:

The **Search Ingestion Service** listens for this event and picks it up.

6. Ingesting Data into Elastic Search:

The **Search Ingestion Service** adds (ingests) the data into **Elastic Search**. We use multiple databases here to store search data, forming the **Elastic Search Cluster**.

Search Process

When a guest searches for a property, the search happens in **Elastic Search**, which is fast and optimized to handle complex queries.

1. Search Request:

When a guest clicks the search button after selecting check-in/out dates, location, price, or typing something like "seaview," a search request goes to the **API Gateway**.

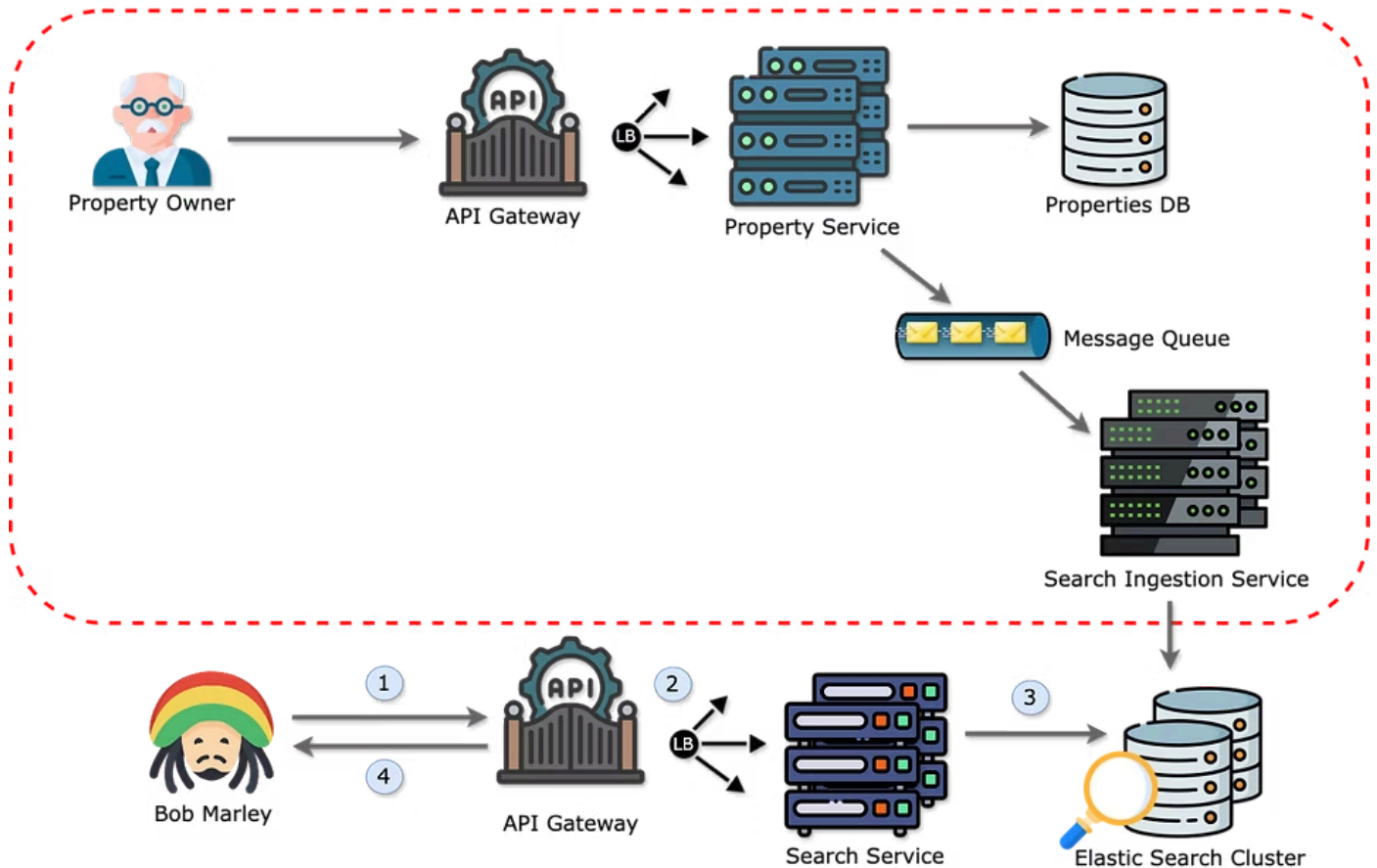
2. Forwarding the Request:

The **API Gateway** sends the request to the **Search Service** through a **Load Balancer**.

3. Search Execution:

The **Search Service** looks for matching properties in **Elastic Search** based on the guest's preferences.

Search Data 'Pre'-Processing



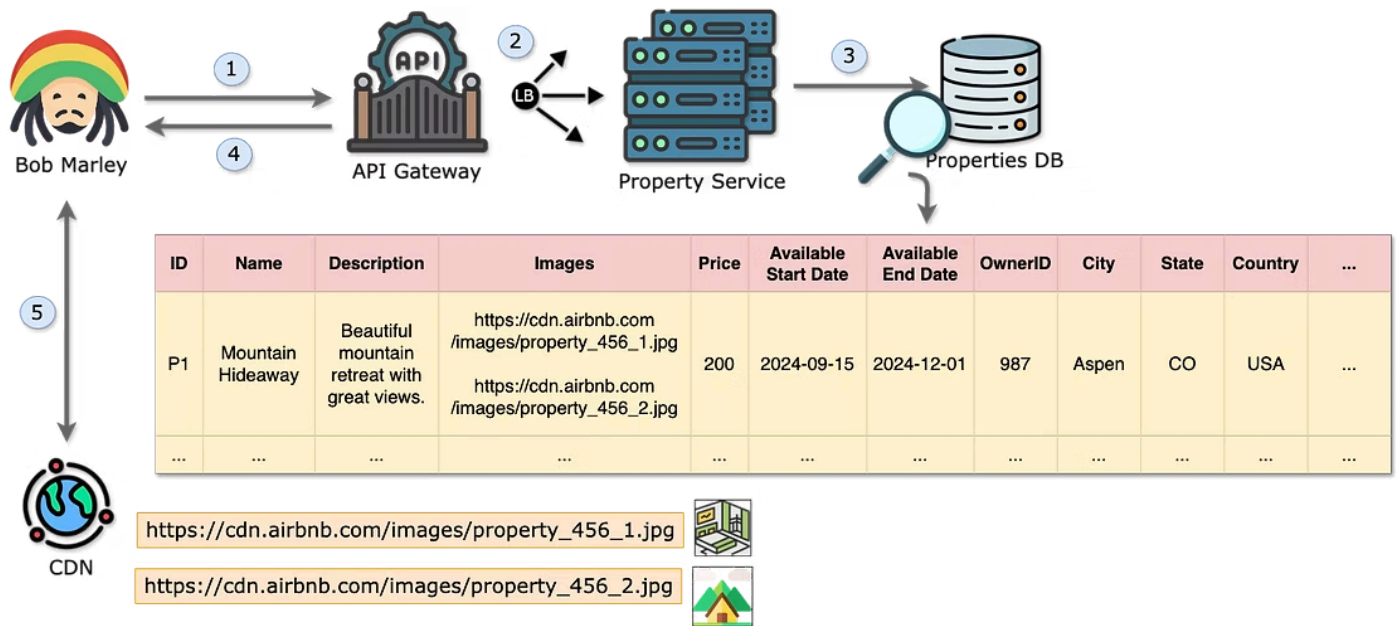
Proactive Data Preparation

Basically, we are preparing the search data in advance (**proactively** & NOT **reactively**) by sending the property data to **Elastic Search** as soon as it's added to the **Properties DB**.

This process is called **Search Data Pre-Processing**, as it ensures the data is ready before any search happens, making it quicker to serve search results.

HIGH LEVEL DESIGN :View Property

Let's now understand how the flow works when a guest clicks on a property from the search results to view its detailed page.



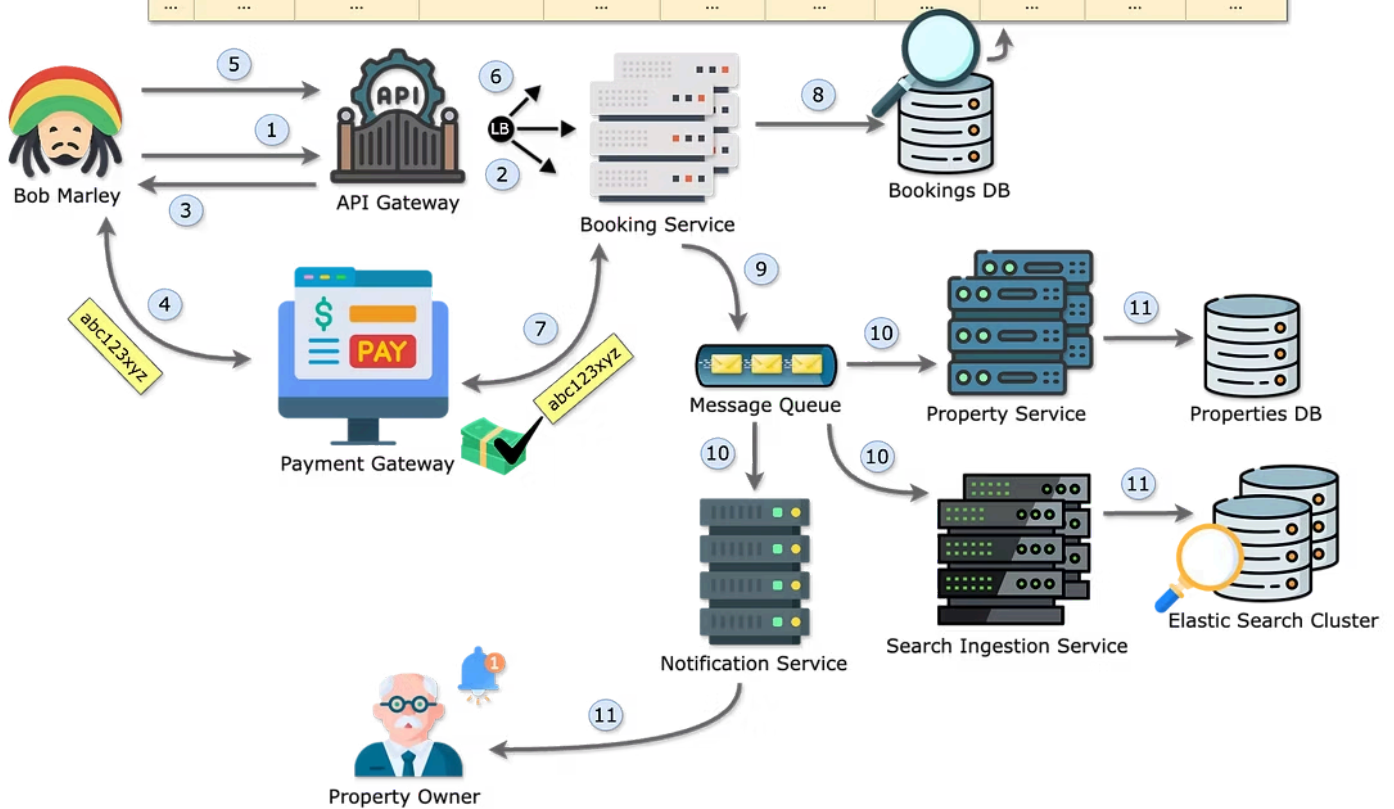
1. As soon as the guest clicks on a property, a **GET** request is sent to the **API Gateway**, requesting all the details of that property.
2. The **API Gateway** forwards this request to the **Property Service** via a **Load Balancer**.
3. The **Property Service** then retrieves all the details of the property from the **Properties Database**.
4. These details are then sent back to the guest.
5. If the property details include image URLs, the guest's browser uses these URLs to download the property images directly from the **CDN**.

This way, all the information, including images, is quickly shown to the guest.

HIGH LEVEL DESIGN :Book Property

Let's break down what happens when a guest actually books a property.

ID	Property ID	Guest ID	Guest Name	Owner ID	Check In	Check Out	Total Price	Payment Token	Booking Status	...
B1	P1	707	Bob Marley	987	2024-10-15	2024-10-20	1000	abc123xyz	Confirmed	...
...	...	...	...	...	...	...	...	...	...	...



Step 1: Guest Initiates Booking

The guest lands on the property's detailed page, enters their check-in and check-out dates along with the number of guests, and clicks the **"Book"** button. As soon as this happens, a request is sent to the **API Gateway**, asking to start the booking process.

Step 2: Request Forwarding

The **API Gateway** forwards this request to the **Booking Service** through the **Load Balancer**.

Step 3: Payment URL Generation

The **Booking Service** then sends a payment URL back to the guest, asking them to complete the payment. This URL takes the guest to a **Payment Gateway**, ensuring the payment is made securely.

Step 4: Payment Completion

Once the guest uses the URL to pay, they get a **payment token**. This token serves as proof of the transaction.

Step 5: Sending Payment Token Back

The guest sends this payment token and the booking details back to the **API Gateway**.

Step 6: Token Validation

The **API Gateway** forwards this information to the **Booking Service** via the **Load Balancer**.

The **Booking Service** sends the guest's payment token to the **Payment Gateway** to confirm if the payment linked to that token was indeed successfully completed.

Step 7: Booking Creation

Once confirmed, the **Booking Service** creates a new booking in the **Bookings Database**.

Step 8: Event Broadcasting

The **Booking Service** also sends an event to the **Message Queue** containing all the booking details.

Step 9: Event Processing

Three services pull this event from the **Message Queue**:

- **Notification Service:**

Uses the owner's ID from the event to notify the owner that their property has been successfully booked.

- **Property Service:**

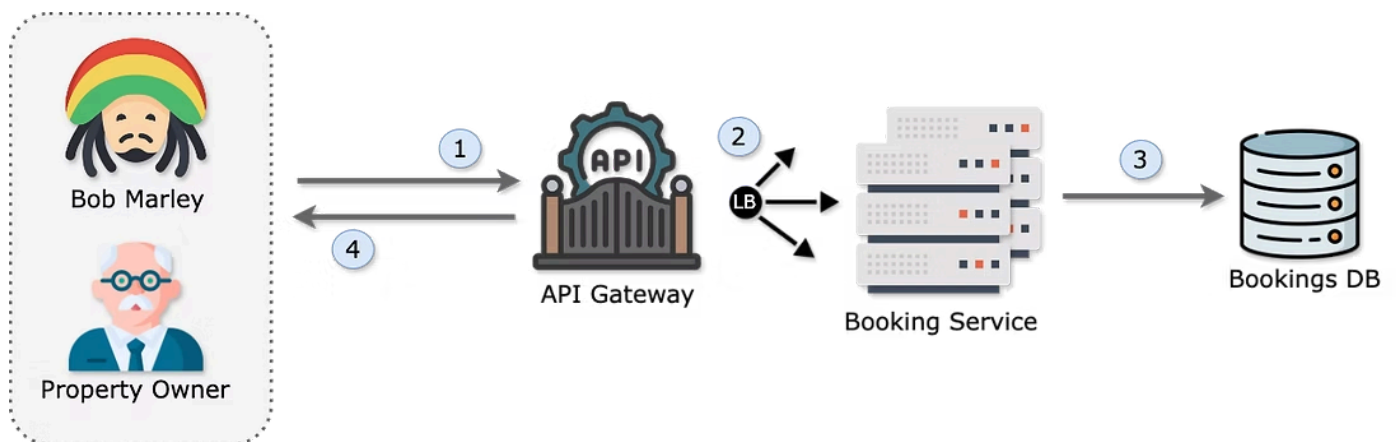
Uses the Check-In & Check-Out dates from the event to update the availability of the property in the **Properties DB**. This helps to block off the booked dates for that property.

- **Search Ingestion Service:**

Uses the Check-In & Check-Out dates from the event to update the availability of the property in the search data in the **Elastic Search Cluster**. This ensures the property won't show up in searches for the dates it's already booked.

HIGH LEVEL DESIGN :View Bookings

Let's break down how the system works when a guest views their bookings. The same flow applies to the property owner when they check all their bookings, so I've explained both in one diagram.



Process Flow

1. When the **Guest** (or **Property Owner**) clicks the "**View Bookings**" button, a **GET** request is sent to the **API Gateway**.
2. The **API Gateway** forwards this request to the **Booking Service** via the **Load Balancer**.
3. The **Booking Service** retrieves all the bookings from the **Bookings Database** for that Guest (or Property Owner).
4. Finally, the bookings are sent back to the **Guest** (or **Property Owner**) for viewing.

DEEP DIVE INSIGHTS: Database Selection

In order to decide the DB type, here are some general guidelines that you can follow. However, remember that it’s not always black and white—a lot depends on the project needs.

Criteria	SQL	NoSQL
Fast Data Access	x	✓
Large Scale	x	✓
Fixed Data Structure	✓	x
Flexible Data Structure	x	✓
Complex Queries	✓	x
Simple Queries	x	✓
Frequent/Evolving Data Changes	x	✓
ACID Transaction Requirements	✓	x

Database Decision Table

► Decision Table (Click to Expand)

Database	Deciding Factors	Decision
Properties DB	• Moderate Latency: Creating or updating property listings doesn't need to be extremely fast. • Fixed Structure: Properties data follow a structured format with predictable fields (name, price, availability, etc.). • Moderate Scale: Estimated ~50 TB storage over 10 years.	SQL
Bookings DB	• ACID Support: Ensures that no other user can book the same property for the same dates. • Moderate Latency: A few seconds of delay is acceptable during booking. • Moderate Scale: Estimated ~1.19 TB storage over 10 years.	SQL

DEEP DIVE INSIGHTS: Data Modeling

Properties DB Schema



```
{
  PropertyID: A unique ID for each property listing.
  Name: The name of the property.
  Description: A brief description of the property.
  Images: URLs to the property images.
  Price: Price per night.
  AvailableStartDates: List of start dates when the property is available.
  AvailableEndDates: List of end dates when the property is no longer available.
  OwnerID: ID of the property owner.
  City: City where the property is located.
  State: State where the property is located.
  Country: Country where the property is located.
  Amenities: ...
  ...
}
```

- **Database Type:** SQL

- **Common Queries:** Fetch a property's details by **PropertyID** when viewing a specific property.
- **Indexing:** **PropertyID**

Note

Since we fetch property details by **PropertyID**, indexing this field will make it quick and efficient to retrieve the record.

Bookings DB Schema



```
{  
  BookingID: A unique identifier for each group.  
  PropertyID: The ID of the property being booked.  
  GuestID: The ID of the guest making the booking.  
  GuestName: Name of the guest.  
  OwnerID: The ID of the property owner.  
  CheckInDate: Date when the guest will check in.  
  CheckOutDate: Date when the guest will check out.  
  TotalPrice: Total price for the stay.  
  PaymentToken: Token received after successful payment.  
  BookingStatus: Status of the booking (e.g., "Confirmed", "Completed", "Cancelled")  
  ...  
}
```

- **Database Type:** SQL
- **Common Queries:** Retrieve bookings for a guest (**GuestID**) or property owner (**OwnerID**) when they want to view their booking details.
- **Indexing:** **GuestID**, **OwnerID**

Note

Indexing **GuestID** and **OwnerID** allows quick access to booking history for both guests and owners.

DEEP DIVE INSIGHTS :Handling Concurrent Booking Updates

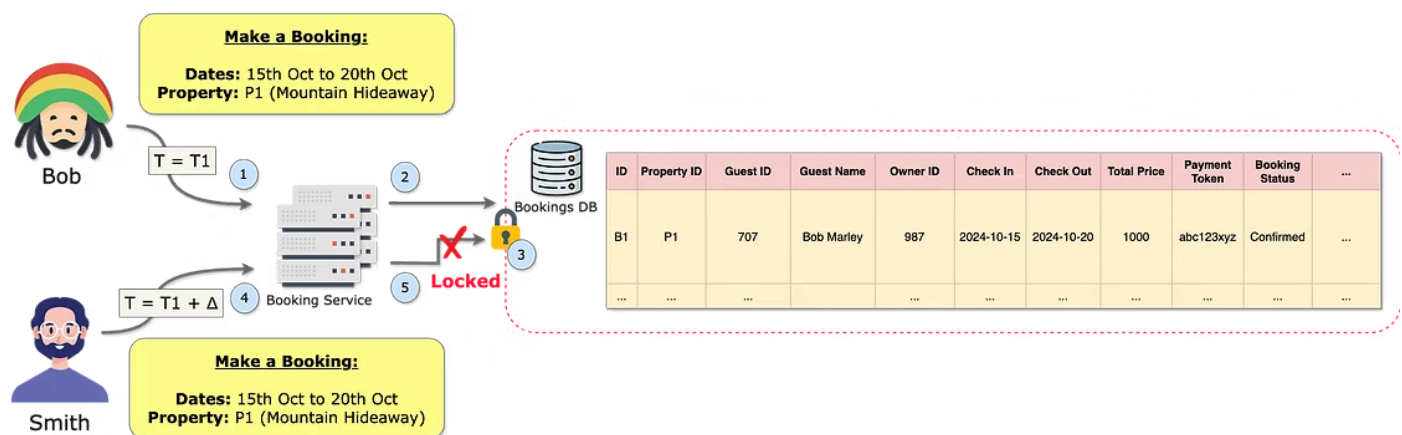
Handling Concurrent Booking Updates

Imagine it's a busy travel season, like a long weekend, and you're planning a trip to Las Vegas. You browse Airbnb and find a highly rated property that you love, and you try to book it. Now, at the same time, someone else in a different part of the U.S. is also planning a trip for the same dates. They also find the same property appealing and try to book it.

So, who actually gets the booking if both try to book at nearly the same time?

This situation is called **concurrent booking**, and Airbnb has to handle this kind of scenario carefully to avoid double bookings.

One way this is managed is through 'locking'. Let's break down what this means with the help of a diagram:



How is Concurrent Booking Handled?

1. Bob Tries to Book

- Bob tries to book a property for **October 15th to October 20th** and sends his request at time $T = T_1$.
- The Booking Service receives Bob's request first and begins creating a booking entry in the Bookings DB.

2. Database Locking

- While creating this entry, the **Bookings DB is locked**, meaning no other booking request can be processed until Bob's booking is fully recorded.

3. Smith Tries to Book

- Around the same time, at $T = T_1 + \Delta$, Smith also tries to book the same property for the same dates.

4. Handling Smith's Request

- Since the database is still locked for Bob's booking, the system prevents Smith's request from being processed at the same time.

By using **database locking**, Airbnb ensures that double bookings are prevented and only one user successfully completes the booking for the property.

Handling Booking Conflicts

Pessimistic Locking

Once Bob's booking is complete, the lock is released. The system now checks if it can process Smith's request. But because Bob has already booked the property for those dates, Smith's booking will be rejected, and duplicate booking will be avoided.

This approach, where we lock the database when someone tries to book, is known as **pessimistic locking**. It is called *pessimistic* because it assumes there will be a conflict. Essentially, it locks the resource (like the property) as soon as someone starts the booking process, operating under the assumption that something will go wrong if we don't lock the resource.

Pros and Cons

- **Pros:** Effective at avoiding conflicts by ensuring that only one request can access a resource at a time.
- **Cons:** In a high-traffic system like Airbnb, where there are thousands of requests every second, this locking approach can slow things down.
If every request locks the database, other users have to wait, which can lead to delays.

Theoretically awesome 🤔, but practically not scalable 😭.

Optimistic Locking

How about we approach the problem with a more optimistic mindset **+++?**

What if we don't lock the resource upfront and let multiple booking requests move forward without locking 🤔?

Here's why this can be a smarter choice.

Why It's Smarter

Think about it. While one request is executing and the database is locked, there might be many other requests in the queue that aren't even conflicting.

For example, someone else could be booking different dates or a different property altogether. Why make them wait?

Conflict Handling

When it comes to conflicting requests—like in the case of Bob and Smith both trying to book the same property for the same dates—we can still process both requests. But just before we finalize them, we'll check for conflicts in the database.

- **If a conflict is found:** Reject the conflicting request and ask the user to try again.
- **If no conflict:** Finalize the booking smoothly.

This approach is more scalable and efficient for high-traffic systems.

Summary

Key Concept

- Allow all requests to go through without locking, so there's no delay.
- Before confirming the booking, perform a quick check for any conflicts in the database.

Benefits

1. Reduced System Delay:

- No locking means the system doesn't choke or slow down, allowing requests to flow smoothly.

2. Parallel Processing:

- Everything can be processed in parallel, and only a double-check is done before finalizing.

3. Efficient Handling of Requests:

- Non-conflicting requests don't have to wait unnecessarily; they proceed without issues.

Why Optimistic Locking?

- **Higher Throughput:**

- More requests are handled in less time, making the system faster.

- **Assumption of Smooth Operations:**

- Optimistic locking assumes that most operations will proceed without conflicts.

- **Conflict Handling:**

- If a conflict arises, it is handled just before the final confirmation.

Ideal for High-Traffic Systems

Optimistic locking is a smart, efficient way to ensure bookings are processed quickly without introducing unnecessary delays while maintaining data integrity.